

Testing

Introduction

You should assume that code that has not been tested does not work. Of course, it is possible to miss test scenarios, but having some testing is always better than none. In this lecture, we'll look at some principles of software testing, specifically as it pertains to unit testing. I could spend an entire course talking about testing, so this lecture will by no means be exhaustive. However, it should be sufficient help you succeed in this class.

Testing should be repeatable

In past classes you have likely been given vague instructions to "test your code", and instructions about user input. But this is not repeatable. Nobody wants to run a program over and over again to test it each time a change is made. Regression testing is done to ensure that changes to code do not break existing functionality. Thus, we will focus on software level tests that can be run as often as you want, with a single command.

Unit Testing

Software unit testing refers to tests that only test small subparts (units) of a software system. There is integration testing for full systems as well, but we won't be covering that in this class (again, I could fill a whole class on testing).

So, what is a unit? It can vary, but for this class each function/method is a unit. We need to create test cases to verify the proper functionality every function or class method we define. Optimally, tests will provide 100% code coverage and handle all edge cases. In the real world, 100% test coverage is impossible to achieve, but the goal is always to get as

close as possible.

Code Coverage

Let's consider this very simple function to perform floating point division of two integers:

divide.hpp

```
double divide(int x, int y) {  
    return (double)x / y;  
}
```

This function has one line of code, so writing a test that achieves 100% coverage of this function is pretty easy. Any test that calls this function will run every line of code in the function, giving us 100% coverage:

test_divide.cpp

```
#include <cassert>  
#include "divide.hpp"  
  
void test_divide_one_by_two() {  
    assert(divide(1, 2) == 0.5);  
}  
  
int main() {  
    test_divide_one_by_two();  
    return 0;  
}
```

This test file has achieved 100% code coverage. Have we fully tested this function? Hopefully, you answered no to this. We have not handled edge cases. We will come back to this example when discussing edge cases in testing.

Consider this next example function that checks if a provided integer is even, and the tests I've created:

even.hpp

```
bool isEven(int x) {  
    if (x % 2 == 0) return true;  
    else return false;  
}
```

test_even.cpp

```
#include <cassert>  
#include "even.hpp"  
  
void test_isEven_zero() {  
    assert(isEven(0));  
}  
  
void test_isEven_two() {  
    assert(isEven(2));  
}  
  
void test_isEven_oneHundred() {  
    assert(isEven(100));  
}  
  
void test_isEven_negativeTwo() {  
    assert(isEven(-2));  
}  
  
int main() {  
    test_isEven_zero();  
    test_isEven_two();  
    test_isEven_oneHundred();  
    test_isEven_negativeTwo();  
    return 0;  
}
```

We have four tests for this function, which check the values 0, 2, 100, and -2. But, do we cover all the paths that our function can take? If we pass 0, the if statement will evaluate to true, and return true. If we pass 2, the if statement's condition will be true, and return true. The same could be said for 100 and -2. This means that the else path of the function is never taken. Therefore, out of two paths, only one is taken,

meaning we only have 50% coverage. We need to add more tests to ensure that the else path is taken:

test_even.cpp

```
#include <cassert>
#include "even.hpp"

void test_isEven_zero() {
    assert(isEven(0));
}

void test_isEven_two() {
    assert(isEven(2));
}

void test_isEven_one_hundred() {
    assert(isEven(100));
}

void test_isEven_negative_two() {
    assert(isEven(-2));
}

// adding tests for odd numbers to cover all paths
void test_isEven_one() {
    assert(!isEven(1));
}

void test_isEven_negative_one() {
    assert(!isEven(-1));
}

void test_isEven_three() {
    assert(!isEven(3));
}

int main() {
    test_isEven_zero();
    test_isEven_two();
    test_isEven_one_hundred();
    test_isEven_negative_two();
    // make sure to call new tests
    test_isEven_one();
    test_isEven_negative_one();
}
```

```
    test_isEven_three();  
    return 0;  
}
```

Now, we have achieved coverage of all potential code paths in our function.

The difficulty in achieving code coverage in tests is proportional to the number of paths in the function. Of course, it is not as simple as just checking for inputs that satisfy/fail every conditional statement. A function may take different paths based on a function that it calls. Therefore, you need to also check which values will cause a function call to generate the right output to take all alternative paths.

```
// need an example that is not too complex...
```

Edge Cases

Let's revisit the `divide` function from earlier: *divide.hpp*

```
double divide(int x, int y) {  
    return (double)x / y;  
}
```

Previously, we defined a single test for it: *test_divide.cpp*

```
#include <cassert>  
#include "divide.hpp"  
  
void test_divide_one_by_two() {  
    assert(divide(1, 2) == 0.5);  
}  
  
int main() {  
    test_divide_one_by_two();  
    return 0;  
}
```

Our function only had one code path, so we achieved 100% coverage. However, we still have only tested one scenario. Let's ponder what can happen when we divide two integers: - We get a whole number as the result - We get a finite decimal for the result - We get an infinite decimal for the result - We get 0 as the result - We can't perform the division, since you cannot divide by 0 So far, we've only tested for one of these cases. Let's add tests for these other cases: *test_divide.cpp*

```
#include <cassert>
#include "divide.hpp"

// We get a finite decimal for the result
void test_divide_one_by_two() {
    assert(divide(1, 2) == 0.5);
}

// We get a whole number as the result
void test_divide_two_by_four() {
    assert(divide(2, 4) == 2);
}

// We get an infinite decimal for the result
void test_divide_one_by_three() {
    assert(divide(1, 3) == 1.0 / 3.0);
}

// We get 0 as the result
void test_divide_zero_by_one_million() {
    assert(divide(0, 1000000) == 0);
}

// We can't perform the division, since you cannot divide by 0
void test_divide_by_zero() {
    // how to test???
    // assert(divide(1, 0) == ???);
}

int main() {
    test_divide_one_by_two();
    test_divide_two_by_four();
    test_divide_one_by_three();
    test_divide_zero_by_one_million();
}
```

```
test_divide_by_zero();  
return 0;  
}
```

As you can see, as our function is currently constructed, we cannot test it for divide by zero. If we tried to divide by zero, our program would crash before we complete the test. Since we want to create defined behavior for this case, let's instead throw an exception in the event we try to divide by zero. Then, we can catch the exception in our function, and error if the exception was not thrown correctly:

divide.hpp

```
#include <stdexcept>  
  
double divide(int x, int y) {  
    if (y == 0) throw std::invalid_argument("Zero passed as divisor. C  
    else return (double)x / y;  
}
```

We've now added a `throw` statement that will raise an `invalid_argument` exception if zero is passed as the divisor. We can pass a custom error message to the `invalid_argument` constructor when we throw the exception. `invalid_argument`, and all standard library exceptions, are found in the `stdexcept` library, so you can note that we have included that library as well. A full list of std exceptions can be found here: <https://en.cppreference.com/w/cpp/header/stdexcept.html>

In this case, you can note that `invalid_argument` made the most sense.

Now, we can modify our test to use a `try...catch...` to catch the correct exception. Note that since I specifically specified in the parenthesis for the catch that I am looking for a

`std::invalid_argument` exception, any other exception will still be unhandled, cause the program to crash, and fail the test:

```
void test_divide_by_zero() {  
    try {  
        divide(1, 0);  
    }  
    catch (std::invalid_argument) {  
        // do nothing, this is what we wanted  
    }  
}
```

We've now achieved, to the best of our knowledge, coverage of all edge cases. Of course, someone may come along later and notice that our testing was insufficient. We've failed to account for underflow or overflow. But, we have still proven that our function works for the set of input that we envisioned it being used for. And, we have these tests set up for regression testing later, should we decide to handle overflow and underflow.